

FR-20 - Mobile Checkout

1. Feature Overview

Feature D selects Mobile Checkout under FR-20. The mobile app uses React Native/Expo and connects to the backend through `API_URL`. The checkout flow uses local state `cart`, `cartTotal`, coupon, token, and sends `POST /api/checkout`.

2. Requirement Summary

FR-20 requires the mobile app to include cart and checkout. FR-08 also applies: only logged-in users can checkout, total is calculated from the cart, UI displays product list, backend recalculates total, and cart is cleared after success. In the mobile code, checkout total is readonly and cart is cleared after success, but the payload sends `items: cart.length > 1 ? cart.slice(0, -1) : cart`, so the last item may be dropped when there are multiple items. Backend currently ignores `items` and trusts `total_amount`.

3. Domain Testing

3.1 Domain Variables and Conditions

Variable / Condition	Description	Valid Domain	Invalid Domain
Mobile login state	<code>user</code> and <code>token</code> in app	Logged in, valid token	Not logged in, empty/invalid token
Mobile cart	Local <code>cart</code> state	At least 1 valid item	Empty, item missing fields, item lost in payload
Quantity	Mobile item quantity	Positive integer, normalized to ≥ 1 when added	0, negative, text, cart input increments incorrectly
Total	<code>cartTotal</code> and final amount	Sum of <code>price * quantity</code>	Manipulated through proxy/API, wrong due to quantity
Coupon	<code>POST /api/apply-coupon</code>	Code exists and conditions are met	Empty, expired, below minimum, usage exceeded
API URL/network	LAN backend connection	Correct IP, backend running	Wrong IP, network lost, API down
Checkout payload	Body sent by mobile	No item lost, total verified	<code>items</code> missing last item, manipulated <code>total_amount</code>
After checkout	Mobile state	Success, cart empty, order history reloaded	Cart cleared at wrong time, order not created, error but cart cleared

3.2 Domain Testing Explanation

1. Identify inputs: login state, local cart, quantity, total, coupon, network, and payload.
2. Divide each variable into valid and invalid domains according to mobile and backend code.
3. Select representatives: not logged in, empty cart, one item, multiple items, `SAVE10/EXPIRED`, wrong network IP.
4. Combine positive and negative cases to cover UI, API, and state behavior.
5. Add API, security, state, and UI cases because mobile uses local state and backend does not verify line items.
6. Review code: mobile checkout shows readonly total but payload uses `cart.slice(0, -1)` for multiple items; mobile profile also sends wrong field `shippingAddress` for address.

3.3 Domain Testing Test Cases

Mobile evidence update: New FR20 screenshots were reviewed on 2026-07-01. Visible UI behavior is judged from screenshots. Hidden request payload and backend recalculation behavior is judged only where source code directly proves the behavior.

TC ID	Technique	Domain Focus	Preconditions	Input Data	Steps	Expected Result	Actual Result	Verdict	Evidence
FR20-DT-01	Domain Testing	Valid mobile checkout	Mobile app running, user logged in, cart has 1 item	iPhone qty 1	Cart -> Checkout -> Confirm	Order created, success shown, cart empty	Screenshot shows checkout success and cart count 0.	Pass	FR20-DT-01
FR20-DT-02	Domain Testing	Not logged in	Cart has item, <code>user=null</code>	One cart item, no logged-in user	Tap checkout	App alerts login required and switches to login	Screenshot shows login-required alert and login screen.	Pass	FR20-DT-02
FR20-DT-03	Domain Testing	Empty cart	User logged in, cart empty	Cart length 0	Open Cart	Empty cart is shown;	Screenshot shows empty cart message	Pass	FR20-DT-03

TC ID	Technique	Domain Focus	Preconditions	Input Data	Steps	Expected Result	Actual Result	Verdict	Evidence
						checkout is not available	and no checkout button.		
FR20-DT-04	Domain Testing	One item	User logged in	Cart 1 item qty 1	Checkout	UI displays exactly 1 item and readonly correct total	Screenshot shows one item with quantity 1 and total 30,000,000 .	Pass	FR20-DT-04
FR20-DT-05	Domain Testing	Multiple items	User logged in	Cart 2-3 items	Checkout	UI displays all items and correct total	Screenshot shows 3 items and total 103,000,000 .	Pass	FR20-DT-05
FR20-DT-06	Domain Testing	Payload loses last item	User logged in, cart >=2 items	Inspect/proxy checkout request	Confirm checkout and capture request	Payload/order must not lose items; backend validates line items	Source review shows mobile sends cart.slice(0, -1) when cart.length > 1 ; backend does not validate line items.	Fail	Source review note; request/response capture still recommended
FR20-DT-07	Domain Testing	Manipulated total through proxy/API	User logged in, cart sum is high	total_amount=1	Send checkout through proxy/API	Backend recalculates or rejects	Source review shows backend inserts supplied total_amount directly without recalculating.	Fail	Source review note; request/response capture still recommended
FR20-DT-08	Domain Testing	Valid coupon	User logged in, total meets minimum	SAVE10 or VIP100	Apply coupon, confirm checkout	Discount/final amount correct, usage recorded after checkout	Screenshot shows SAVE10 applied but total increases from 103,000,000 to 1,030,000,000 .	Fail	FR20-DT-08
FR20-DT-09	Domain Testing	Invalid coupon	User logged in	EXPIRED , non-existing code, below-minimum total	Apply coupon	Error shown, final amount unchanged	Screenshot shows invalid coupon error and total remains 103,000,000 .	Pass	FR20-DT-09
FR20-DT-10	Domain Testing	Network/API error	Backend stopped or API_URL wrong IP	API unavailable or unreachable URL	Confirm checkout	App shows error and cart is not cleared	Screenshot shows Network request failed and cart count remains 3 .	Pass	FR20-DT-10
FR20-DT-11	Domain Testing	Cart cleared after success	User logged in, cart has item	Successful checkout	Return to Cart	Mobile cart empty and order history reloaded	Screenshot confirms cart empty after checkout; source review shows successful checkout calls setCart([]) and fetchOrders(token) .	Pass	FR20-DT-11
FR20-DT-12	Domain Testing	Abnormal quantity in mobile cart	User logged in	Enter qty 0/text in cart input	Observe quantity and checkout	Quantity is safely normalized; total is not wrong	Screenshot shows quantity 0 entered on product detail; source review shows addToCart normalizes zero/text quantity to 1 , preventing a wrong checkout total from that path.	Pass	FR20-DT-12

3.4 Review Notes for New Screenshot Evidence

TC ID	Previous Status	Final Judgment	Reason	Evidence
FR20-DT-01	To be executed	Pass	Checkout success and empty cart count are visible.	FR20-DT-01

TC ID	Previous Status	Final Judgment	Reason	Evidence
FR20-DT-02	To be executed	Pass	Login-required alert and login screen are visible.	FR20-DT-02
FR20-DT-03	To be executed	Pass	Empty cart message is visible and no checkout button is shown.	FR20-DT-03
FR20-DT-04	To be executed	Pass	One item and correct subtotal are visible.	FR20-DT-04
FR20-DT-05	To be executed	Pass	Three items and correct total are visible.	FR20-DT-05
FR20-DT-06	To be executed	Fail	Source code omits the last cart item from checkout payload when cart has more than one item.	Source review note; request/response capture still recommended
FR20-DT-07	To be executed	Fail	Backend stores client-supplied <code>total_amount</code> without recalculation.	Source review note; request/response capture still recommended
FR20-DT-08	To be executed	Fail	Valid coupon calculation increases the payable total instead of reducing it in the captured run.	FR20-DT-08
FR20-DT-09	To be executed	Pass	Invalid coupon error is visible and total remains unchanged.	FR20-DT-09
FR20-DT-10	To be executed	Pass	Network failure alert is visible and cart count remains unchanged.	FR20-DT-10
FR20-DT-11	To be executed	Pass	Cart empty state is visible, and source review shows the success path reloads orders with <code>fetchOrders(token)</code> .	FR20-DT-11
FR20-DT-12	To be executed	Pass	Product-detail quantity <code>0</code> is visible, and source review shows add-to-cart normalizes invalid quantities to <code>1</code> .	FR20-DT-12

4. Boundary Value Analysis

4.1 Boundary Variables

Variable	Boundary Rule	Below Boundary	On Boundary	Above Boundary
Cart item count	Checkout requires ≥ 1 item	0	1	2
Quantity	Minimum valid quantity is 1	0	1	2
Total amount	Must equal cart sum	sum-1	sum	sum+1
Coupon minimum amount	Must reach threshold	min-1	min	min+1
Token	Valid token required	No token	Valid token	Invalid token
Network/API result	Local state must not be lost on error	Error before send	Success	Error after response

4.2 BVA Explanation

1. Identify item-count, quantity, total, coupon-threshold, token, and network boundaries.
2. Select below/on/above boundary values to observe mobile state and API response.
3. Create tests around 0/1/2 items because mobile code has special logic when `cart.length > 1`.
4. Include valid and invalid boundaries.
5. Review code: 2 items is an important boundary because payload starts using `slice(0, -1)`.

4.2 BVA Test Cases

Mobile evidence update: New FR20 screenshots were reviewed on 2026-07-01. Boundary cases with visible UI evidence were updated. Boundary cases requiring payload capture or complete coupon threshold/API runs remain pending unless source evidence confirms the defect.

TC ID	Technique	Domain Focus	Preconditions	Input Data	Steps	Expected Result	Actual Result	Verdict	Evidence
FR20-BVA-01	Boundary Value Analysis	0 items	User logged in, cart empty	Cart length 0	Open Cart/Checkout	Checkout is not available	Screenshot shows empty cart and no checkout button.	Pass	FR20-BVA-01

TC ID	Technique	Domain Focus	Preconditions	Input Data	Steps	Expected Result	Actual Result	Verdict	Evidence
FR20-BVA-02	Boundary Value Analysis	1 item	User logged in	Cart length 1	Checkout	Successful if API OK; payload contains 1 item	Screenshot shows successful checkout; source review indicates the full cart is sent when length is 1.	Pass	FR20-BVA-02
FR20-BVA-03	Boundary Value Analysis	2 items	User logged in	Cart length 2	Checkout and inspect payload	No item is lost	Source review shows carts with more than one item send <code>cart.slice(0, -1)</code> , so the last item is omitted from payload.	Fail	FR20-BVA-03 ; request/response capture still recommended
FR20-BVA-04	Boundary Value Analysis	Quantity 0/1/2	User logged in	0, 1, 2	Modify quantity in mobile cart	0 is normalized/blocked; 1/2 calculated correctly	Source review shows cart quantity editing changes any positive parsed value to <code>parsed + 1</code> , so entering 1 becomes 2 and entering 2 becomes 3.	Fail	FR20-BVA-04
FR20-BVA-05	Boundary Value Analysis	Coupon threshold	User logged in	<code>SAVE10</code> with 299999, 300000, 300001	Apply coupon	According to SRS, on/above minimum is valid; below minimum is rejected	To be executed	To be executed	To be generated
FR20-BVA-06	Boundary Value Analysis	Total around sum	User logged in, sum=30000000	29999999, 30000000, 30000001 via proxy/API	Checkout	Only correct/recalculated total is accepted	Source review shows backend accepts supplied <code>total_amount</code> without recalculating.	Fail	Source review note; request/response capture still recommended

4.4 Review Notes for New Screenshot Evidence

TC ID	Previous Status	Final Judgment	Reason	Evidence
FR20-BVA-01	To be executed	Pass	Empty cart boundary has no checkout action visible.	FR20-BVA-01
FR20-BVA-02	To be executed	Pass	One-item checkout succeeds, and source review indicates the full cart is used for a one-item payload.	FR20-BVA-02
FR20-BVA-03	To be executed	Fail	Source review shows the last item is omitted when cart length is greater than 1.	Source review note; request/response capture still recommended
FR20-BVA-04	To be executed	Fail	Source review shows the cart quantity input increments positive values by one instead of preserving 1 and 2.	FR20-BVA-04
FR20-BVA-06	To be executed	Fail	Backend accepts boundary totals from request body without recalculation.	Source review note; request/response capture still recommended

5. Execution Summary

Designed	Executed / Reviewed	Pass	Fail	Needs Review	To be executed
18	17	11	6	0	1

Execution evidence includes new mobile screenshots under [evidence/screenshots/](#) and source review for hidden checkout payload, total, order reload, and quantity behavior.

6. AI Gap Analysis

6.1 AI-Suggested Cases

AI commonly suggests successful mobile checkout, unauthenticated checkout, empty cart, one/multiple items, and valid/invalid coupons.

6.2 Missing / Weak AI Cases

AI may miss `cart.slice(0, -1)`, hard-coded LAN `API_URL`, network error, local state versus backend mismatch, and backend ignoring `items`.

6.3 Why AI Might Miss Them

These issues are hidden in mobile implementation details and are visible only after reading `App.js`. If only the requirement is used, AI may assume mobile sends the full cart and backend verifies totals.

6.4 Human Corrections

After reading source code and reviewing the new screenshots, Feature D remained Mobile Checkout and added cases for multiple items/incomplete payload, manipulated total, network error, coupon boundary, and cart clearing after success. Screenshot-supported UI cases were updated, source-confirmed backend/payload defects were marked as failures, and the previously incomplete review cases were resolved through source review.

7. Bugs or Remaining Review

The new screenshot evidence and source review confirm three FR-20 mobile/backend checkout defects.

Related Test Case	Verdict	Bug Summary	Evidence
FR20-DT-06, FR20-BVA-03	Fail	Multi-item checkout payload omits the last cart item because mobile sends <code>cart.slice(0, -1)</code> when cart length is greater than 1.	Source review note; request/response capture still recommended
FR20-DT-07, FR20-BVA-06	Fail	Backend checkout trusts client-supplied <code>total_amount</code> and does not recalculate from cart items.	Source review note; request/response capture still recommended
FR20-DT-08	Fail	Valid <code>SAVE10</code> coupon produced an incorrect final amount, increasing <code>103,000,000</code> to <code>1,030,000,000</code> .	FR20-DT-08

FR20-BVA-05 still has no execution evidence and remains `To be executed`.

8. Remaining Manual Review

Item	Description	Required Evidence
FR20-BVA-05	Coupon threshold values have no evidence.	Screenshot or API log for below, on, and above threshold.